

```
In [15]: # Arkaprava Roy
# 17.09.2025
```

```
In [16]: import pandas as pd
import matplotlib.pyplot as plt
```

```
In [17]: #Data Prep
```

```
In [18]: purchase_behavior = pd.read_csv('QVI_purchase_behaviour.csv')
transactions = pd.read_excel('QVI_transaction_data.xlsx')
```

```
In [19]: print(purchase_behavior.head()) # top 5 rows of purchase behaviour
print(transactions.head()) # top 5 rows of transactions
```

	LYLTY_CARD_NBR		LIFESTAGE		PREMIUM_CUSTOMER
0	1000	YOUNG	SINGLES/COUPLES		Premium
1	1002	YOUNG	SINGLES/COUPLES		Mainstream
2	1003		YOUNG FAMILIES		Budget
3	1004	OLDER	SINGLES/COUPLES		Mainstream
4	1005	MIDAGE	SINGLES/COUPLES		Mainstream

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	\
0	43390	1	1000	1	5	
1	43599	1	1307	348	66	
2	43605	1	1343	383	61	
3	43329	2	2373	974	69	
4	43330	2	2426	1038	108	

		PROD_NAME	PROD_QTY	TOT_SALES
0	Natural Chip	Compny SeaSalt175g	2	6.0
1		CCs Nacho Cheese 175g	3	6.3
2	Smiths Crinkle Cut	Chips Chicken 170g	2	2.9
3	Smiths Chip Thinly	S/Cream&Onion 175g	5	15.0
4	Kettle Tortilla	ChpsHny&Jlpno Chili 150g	3	13.8

```
In [20]: transactions['DATE'] = pd.to_datetime(transactions['DATE'], origin='1899-12-30', unit='D')
transactions['DATE'].head()
```

Out[20]: 0 2018-10-17
1 2019-05-14
2 2019-05-20
3 2018-08-17
4 2018-08-18
Name: DATE, dtype: datetime64[ns]

```
In [21]: transactions['PACK_SIZE'] = transactions['PROD_NAME'].str.extract(r'(\d+)').astype(float)
transactions['BRAND'] = transactions['PROD_NAME'].str.split().str[0]
```

```
In [22]: transactions[['PROD_QTY', 'TOT_SALES']].describe()
```

Out[22]:

	PROD_QTY	TOT_SALES
count	264836.000000	264836.000000
mean	1.907309	7.304200
std	0.643654	3.083226
min	1.000000	1.500000
25%	2.000000	5.400000
50%	2.000000	7.400000
75%	2.000000	9.200000
max	200.000000	650.000000

```
In [23]: transactions = transactions[transactions['PROD_QTY'] < 10]
transactions = transactions[transactions['TOT_SALES'] < 100]
```

```
In [24]: transactions[['PROD_QTY', 'TOT_SALES']].describe()
```

Out[24]:

	PROD_QTY	TOT_SALES
count	264834.000000	264834.000000
mean	1.905813	7.299346
std	0.343436	2.527241
min	1.000000	1.500000
25%	2.000000	5.400000
50%	2.000000	7.400000
75%	2.000000	9.200000
max	5.000000	29.500000

In [25]:

transactions['PACK_SIZE'] = transactions['PROD_NAME'].str.extract(r'(\d+)').astype(float)
transactions['BRAND'] = transactions['PROD_NAME'].str.split().str[0]

In [26]:

merged = pd.merge(transactions, purchase_behavior, on='LYLTY_CARD_NBR', how='inner')

In [27]:

#Data Cleaning

In [28]:

merged.isnull().sum()

Out[28]:

DATE 0
STORE_NBR 0
LYLTY_CARD_NBR 0
TXN_ID 0
PROD_NBR 0
PROD_NAME 0
PROD_QTY 0
TOT_SALES 0
PACK_SIZE 0
BRAND 0
LIFESTAGE 0
PREMIUM_CUSTOMER 0
dtype: int64

In [29]:

merged.duplicated().sum()

Out[29]:

1

In [30]:

merged = merged.drop_duplicates()

In [31]:

merged.duplicated().sum()

Out[31]:

0

In [32]:

merged.info()

<class 'pandas.core.frame.DataFrame'>
Index: 264833 entries, 0 to 264833
Data columns (total 12 columns):
Column Non-Null Count Dtype
--- ---
0 DATE 264833 non-null datetime64[ns]
1 STORE_NBR 264833 non-null int64
2 LYLTY_CARD_NBR 264833 non-null int64
3 TXN_ID 264833 non-null int64
4 PROD_NBR 264833 non-null int64
5 PROD_NAME 264833 non-null object
6 PROD_QTY 264833 non-null int64
7 TOT_SALES 264833 non-null float64
8 PACK_SIZE 264833 non-null float64
9 BRAND 264833 non-null object
10 LIFESTAGE 264833 non-null object
11 PREMIUM_CUSTOMER 264833 non-null object
dtypes: datetime64[ns](1), float64(2), int64(5), object(4)
memory usage: 26.3+ MB

In [33]:

merged.describe()

Out[33]:

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_QTY	TOT_SALES	PACK_SI
count	264833	264833.000000	2.648330e+05	2.648330e+05	264833.000000	264833.000000	264833.000000	264833.0000
mean	2018-12-30 00:52:39.666657792	135.079529	1.355489e+05	1.351577e+05	56.583598	1.905812	7.299351	182.4255
min	2018-07-01 00:00:00	1.000000	1.000000e+03	1.000000e+00	1.000000	1.000000	1.500000	70.0000
25%	2018-09-30 00:00:00	70.000000	7.002100e+04	6.760000e+04	28.000000	2.000000	5.400000	150.0000
50%	2018-12-30 00:00:00	130.000000	1.303570e+05	1.351370e+05	56.000000	2.000000	7.400000	170.0000
75%	2019-03-31 00:00:00	203.000000	2.030940e+05	2.027000e+05	85.000000	2.000000	9.200000	175.0000
max	2019-06-30 00:00:00	272.000000	2.373711e+06	2.415841e+06	114.000000	5.000000	29.500000	380.0000
std	NaN	76.784189	8.058003e+04	7.813305e+04	32.826498	0.343437	2.527244	64.3252

In [34]:

```
merged['PACK_SIZE'] = merged['PROD_NAME'].str.extract(r'(\d+)').astype(int)
```

In [35]:

```
merged['BRAND'] = merged['PROD_NAME'].str.split().str[0]
```

In [37]:

```
merged.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALES'].sum()
```

Out[37]:

LIFESTAGE	PREMIUM_CUSTOMER	
MIDAGE SINGLES/COUPLES	Budget	35514.80
	Mainstream	90803.85
	Premium	58432.65
NEW FAMILIES	Budget	21928.45
	Mainstream	17013.90
	Premium	11491.10
OLDER FAMILIES	Budget	168363.25
	Mainstream	103445.55
	Premium	80658.40
OLDER SINGLES/COUPLES	Budget	136769.80
	Mainstream	133393.80
	Premium	132257.15
RETIREEES	Budget	113147.80
	Mainstream	155677.05
	Premium	97646.05
YOUNG FAMILIES	Budget	139345.85
	Mainstream	92788.75
	Premium	84025.50
YOUNG SINGLES/COUPLES	Budget	61141.60
	Mainstream	157621.60
	Premium	41642.10

Name: TOT_SALES, dtype: float64

In [38]:

```
#Grouping
```

In [39]:

```
qty_segment = merged.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['PROD_QTY'].sum().unstack()
print(qty_segment)
```

PREMIUM_CUSTOMER	Budget	Mainstream	Premium
LIFESTAGE			
MIDAGE SINGLES/COUPLES	9496	22699	15526
NEW FAMILIES	5571	4319	2957
OLDER FAMILIES	45065	27756	21771
OLDER SINGLES/COUPLES	35220	34997	33984
RETIREEES	28764	40518	24884
YOUNG FAMILIES	37111	25044	22406
YOUNG SINGLES/COUPLES	16671	38632	11331

In [40]:

```
avg_spend_segment = merged.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALES'].mean().unstack()
print(avg_spend_segment)
```

PREMIUM_CUSTOMER	Budget	Mainstream	Premium
LIFESTAGE			
MIDAGE SINGLES/COUPLES	7.074661	7.647284	7.112056
NEW FAMILIES	7.297321	7.317806	7.231655
OLDER FAMILIES	7.269570	7.262395	7.208079
OLDER SINGLES/COUPLES	7.430315	7.282116	7.449848
RETIREEES	7.443445	7.252262	7.456174
YOUNG FAMILIES	7.287201	7.189025	7.266756
YOUNG SINGLES/COUPLES	6.615624	7.558339	6.629852

```
In [41]: top_brands = merged.groupby('BRAND')['TOT_SALES'].sum().sort_values(ascending=False).head(10)
print(top_brands)
```

```
BRAND
Kettle      390239.8
Smiths      210070.8
Doritos     201538.9
Pringles    177655.5
Old          90785.1
Thins       88852.5
Twisties    81522.1
Tostitos    79789.6
Infuzions   76247.6
Cobs        70569.8
Name: TOT_SALES, dtype: float64
```

```
In [42]: top_packsize = merged.groupby('PACK_SIZE')['TOT_SALES'].sum().sort_values(ascending=False).head(10)
print(top_packsize)
```

```
PACK_SIZE
175      485431.4
150      304288.5
134      177655.5
110      162765.4
170      146673.0
330      136794.3
300      113330.6
165      101360.6
380       75419.6
270       55425.4
Name: TOT_SALES, dtype: float64
```

```
In [43]: brand_by_segment = merged.groupby(['LIFESTAGE', 'BRAND'])['TOT_SALES'].sum().sort_values(ascending=False).head(20)
print(brand_by_segment)
```

```
LIFESTAGE      BRAND
OLDER SINGLES/COUPLES  Kettle      83862.6
RETIREEES           Kettle      76914.8
OLDER FAMILIES       Kettle      65984.0
YOUNG FAMILIES       Kettle      60033.0
YOUNG SINGLES/COUPLES Kettle      54241.0
OLDER SINGLES/COUPLES Doritos     42525.7
                   Smiths      42410.2
OLDER FAMILIES       Smiths      41902.1
RETIREEES           Doritos     39245.4
MIDAGE SINGLES/COUPLES Kettle     38358.2
RETIREEES           Smiths      37625.4
OLDER SINGLES/COUPLES Pringles    37603.1
YOUNG FAMILIES       Smiths      36779.7
OLDER FAMILIES       Doritos     35140.9
RETIREEES           Pringles    34898.4
YOUNG FAMILIES       Doritos     31938.7
OLDER FAMILIES       Pringles    30584.2
YOUNG SINGLES/COUPLES Doritos     28004.3
YOUNG FAMILIES       Pringles    27498.4
YOUNG SINGLES/COUPLES Smiths      26614.3
Name: TOT_SALES, dtype: float64
```

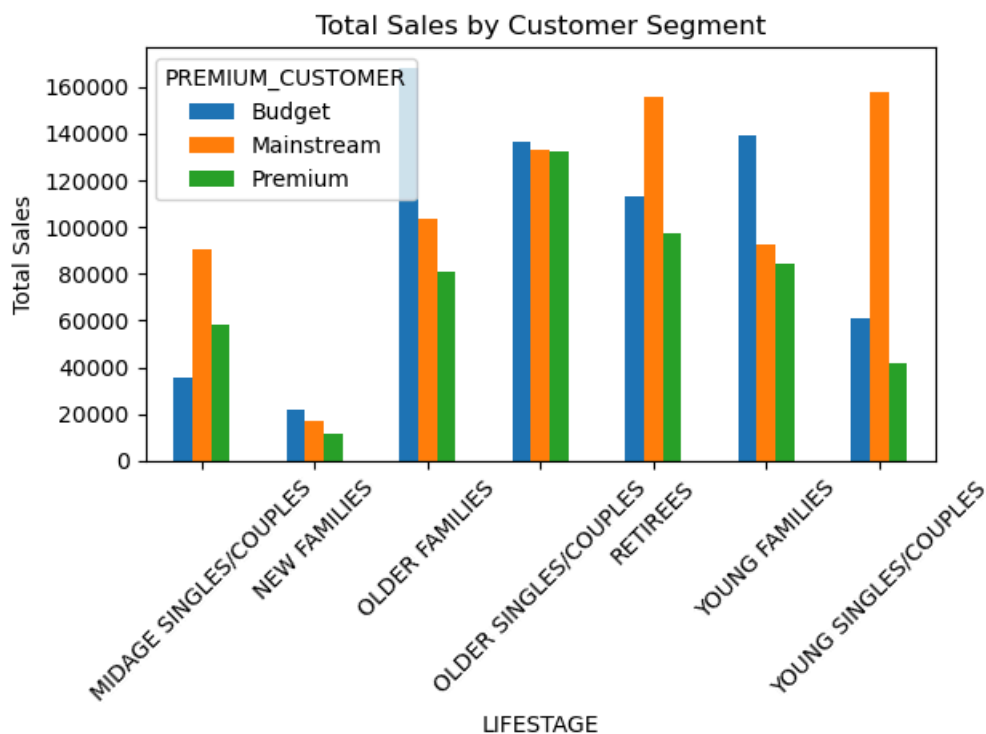
```
In [44]: #Visualization & Plotting
```

```
In [45]: import matplotlib.pyplot as plt

sales_segment = merged.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALES'].sum().unstack()

plt.figure(figsize=(12,6))
sales_segment.plot(kind='bar')
plt.ylabel('Total Sales')
plt.title('Total Sales by Customer Segment')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

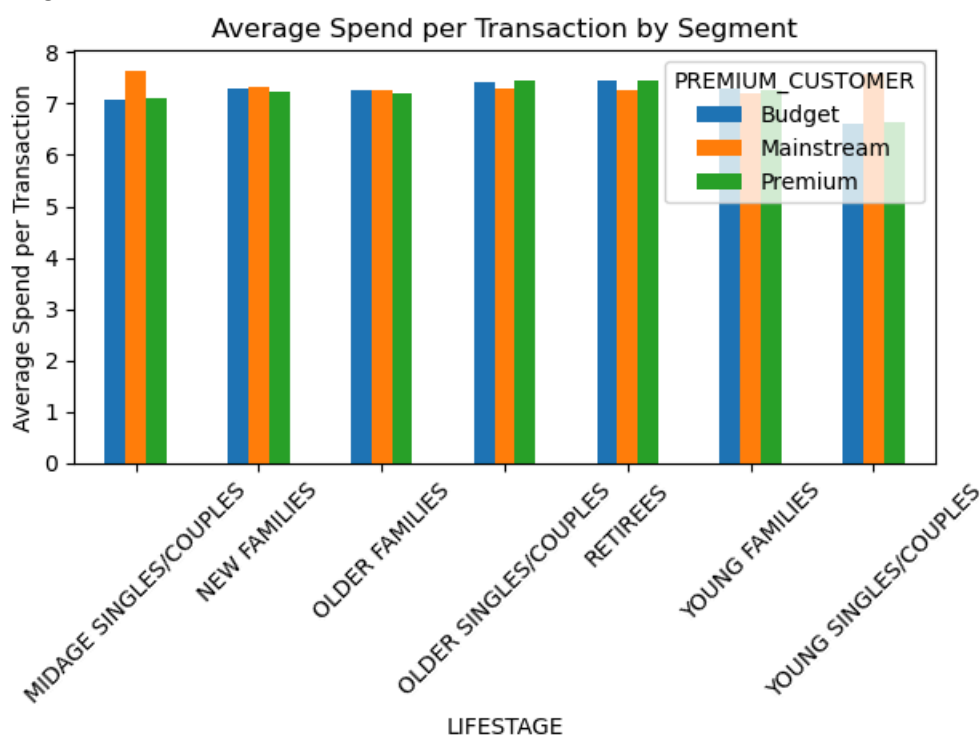
<Figure size 1200x600 with 0 Axes>



```
In [46]: avg_spend_segment = merged.groupby(['LIFESTAGE', 'PREMIUM_CUSTOMER'])['TOT_SALES'].mean().unstack()

plt.figure(figsize=(12,6))
avg_spend_segment.plot(kind='bar')
plt.ylabel('Average Spend per Transaction')
plt.title('Average Spend per Transaction by Segment')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

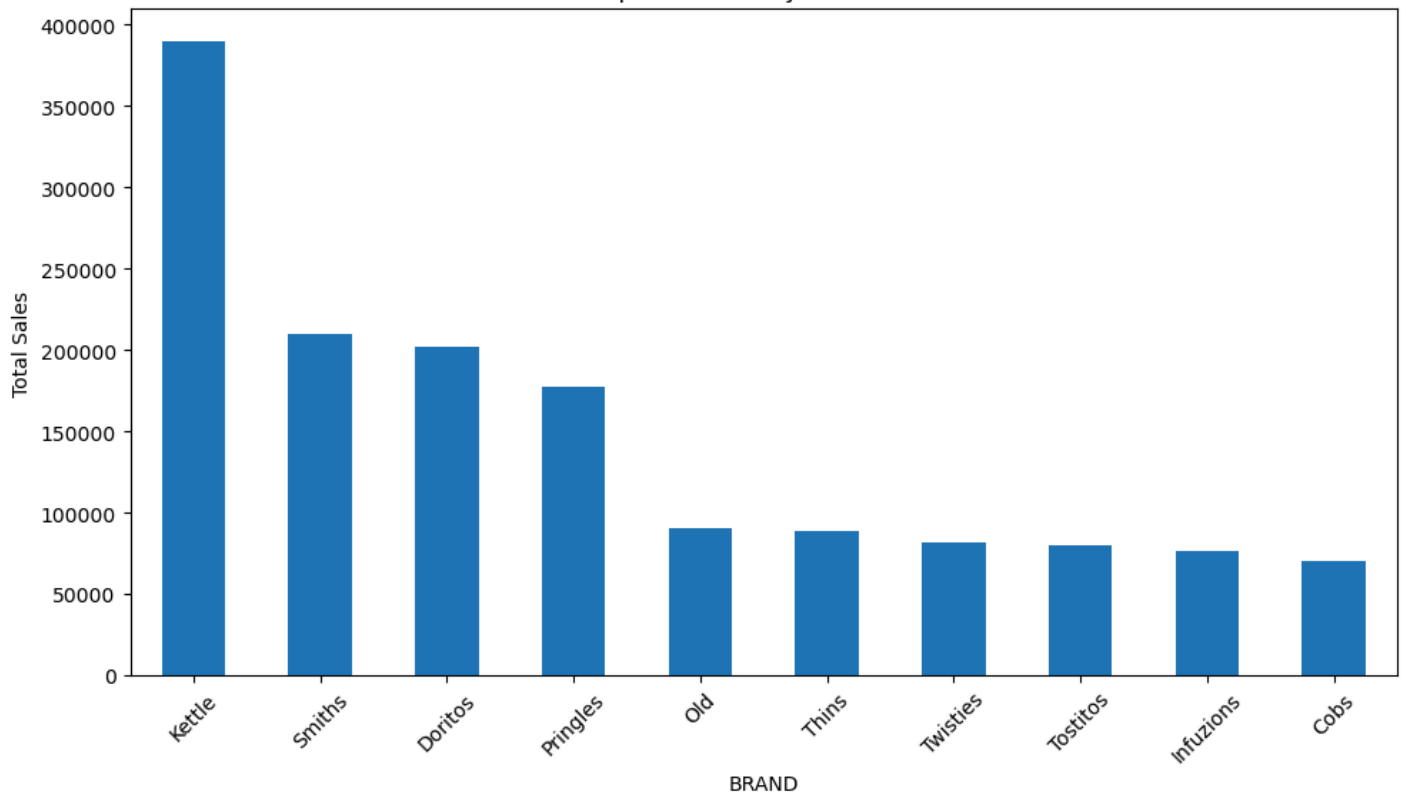
<Figure size 1200x600 with 0 Axes>



```
In [47]: top_brands = merged.groupby('BRAND')['TOT_SALES'].sum().sort_values(ascending=False).head(10)

plt.figure(figsize=(10,6))
top_brands.plot(kind='bar')
plt.ylabel('Total Sales')
plt.title('Top 10 Brands by Total Sales')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

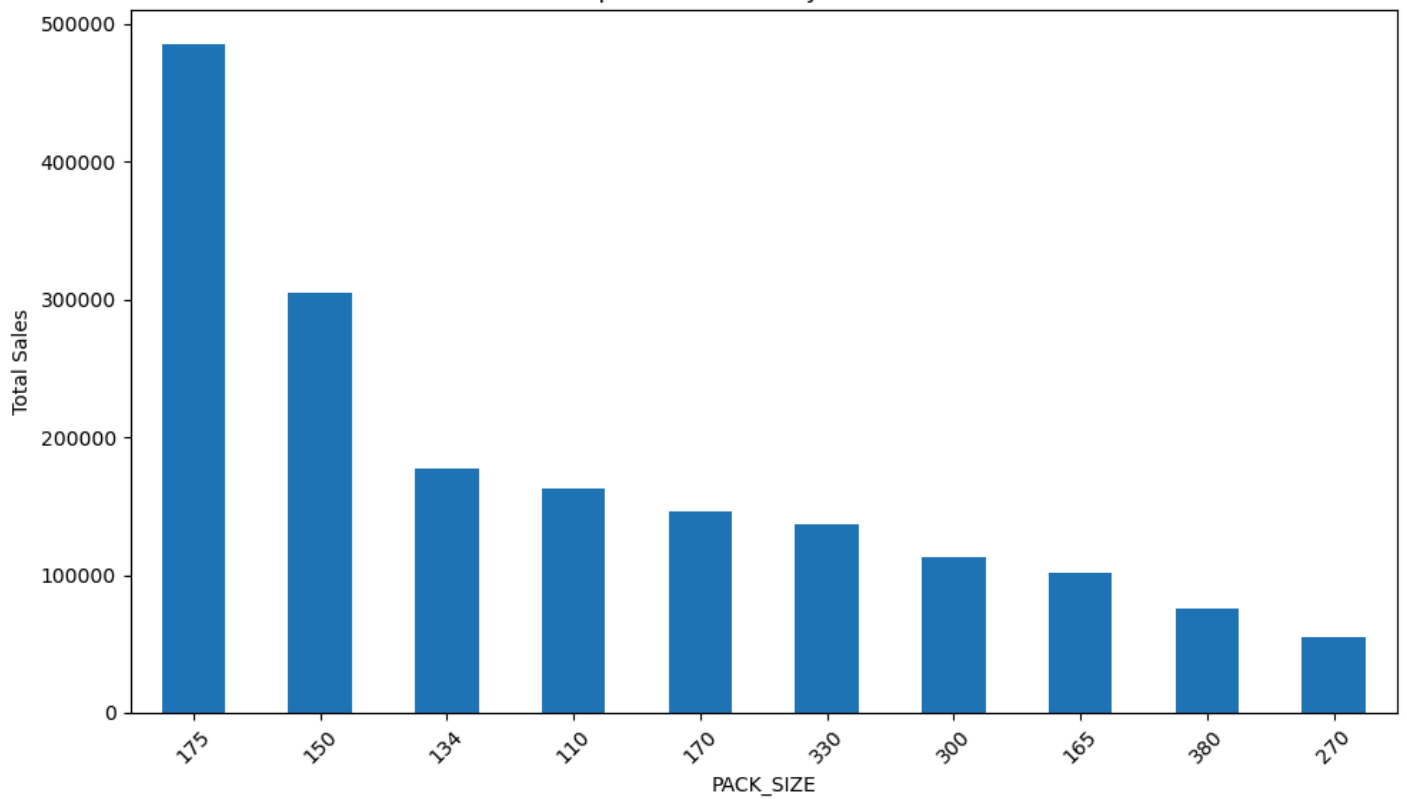
Top 10 Brands by Total Sales



```
In [48]: top_packsize = merged.groupby('PACK_SIZE')['TOT_SALES'].sum().sort_values(ascending=False).head(10)
```

```
plt.figure(figsize=(10,6))
top_packsize.plot(kind='bar')
plt.ylabel('Total Sales')
plt.title('Top 10 Pack Sizes by Total Sales')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

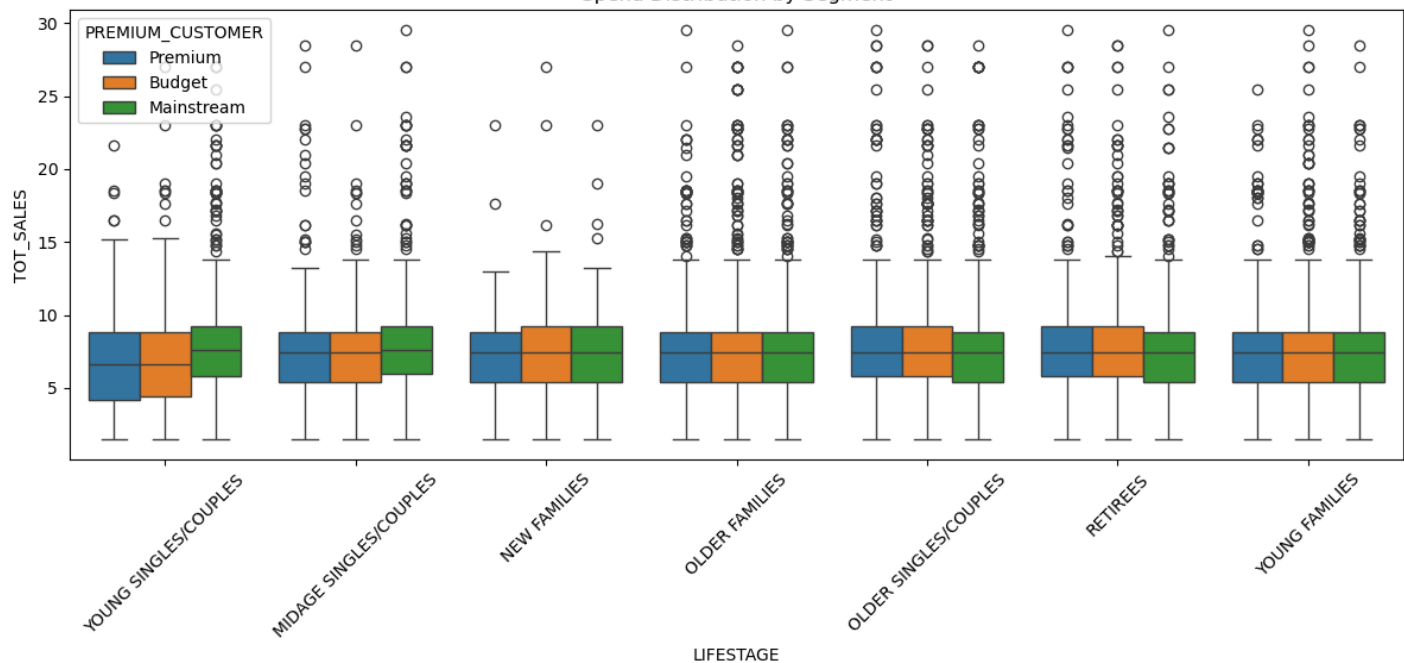
Top 10 Pack Sizes by Total Sales



```
In [49]: import seaborn as sns
```

```
plt.figure(figsize=(12,6))
sns.boxplot(data=merged, x='LIFESTAGE', y='TOT_SALES', hue='PREMIUM_CUSTOMER')
plt.title('Spend Distribution by Segment')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

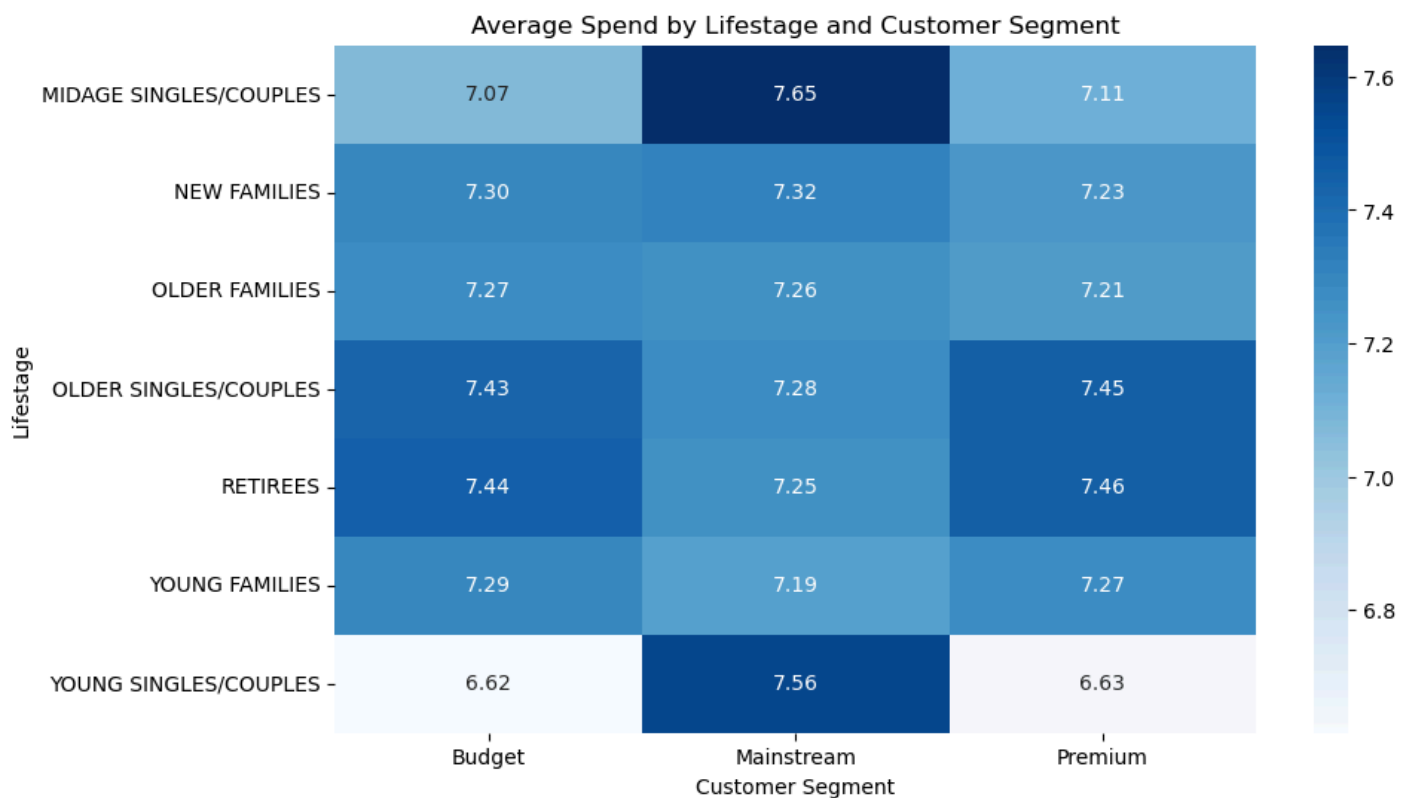
Spend Distribution by Segment



```
In [58]: import seaborn as sns
import numpy as np

# Pivot to create a matrix
heatmap_data = merged.pivot_table(
    index='LIFESTAGE',
    columns='PREMIUM_CUSTOMER',
    values='TOT_SALES',
    aggfunc='mean'
)

plt.figure(figsize=(10,6))
sns.heatmap(heatmap_data, annot=True, fmt=".2f", cmap='Blues')
plt.title('Average Spend by Lifestage and Customer Segment')
plt.xlabel('Customer Segment')
plt.ylabel('Lifestage')
plt.show()
```

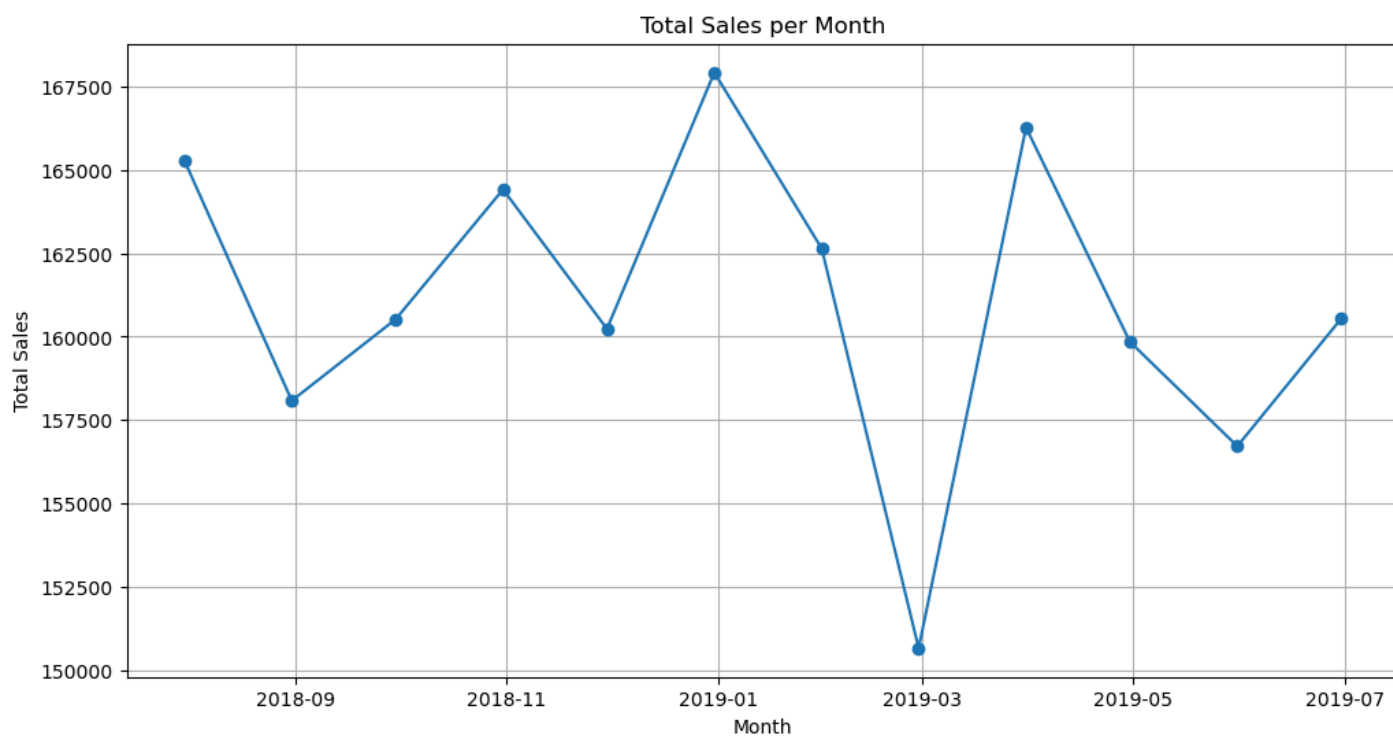


```
In [65]: import pandas as pd
import matplotlib.pyplot as plt

merged['DATE'] = pd.to_datetime(merged['DATE'])

# Group by month and sum sales
monthly_sales = merged.groupby(pd.Grouper(key='DATE', freq='ME'))['TOT_SALES'].sum()
```

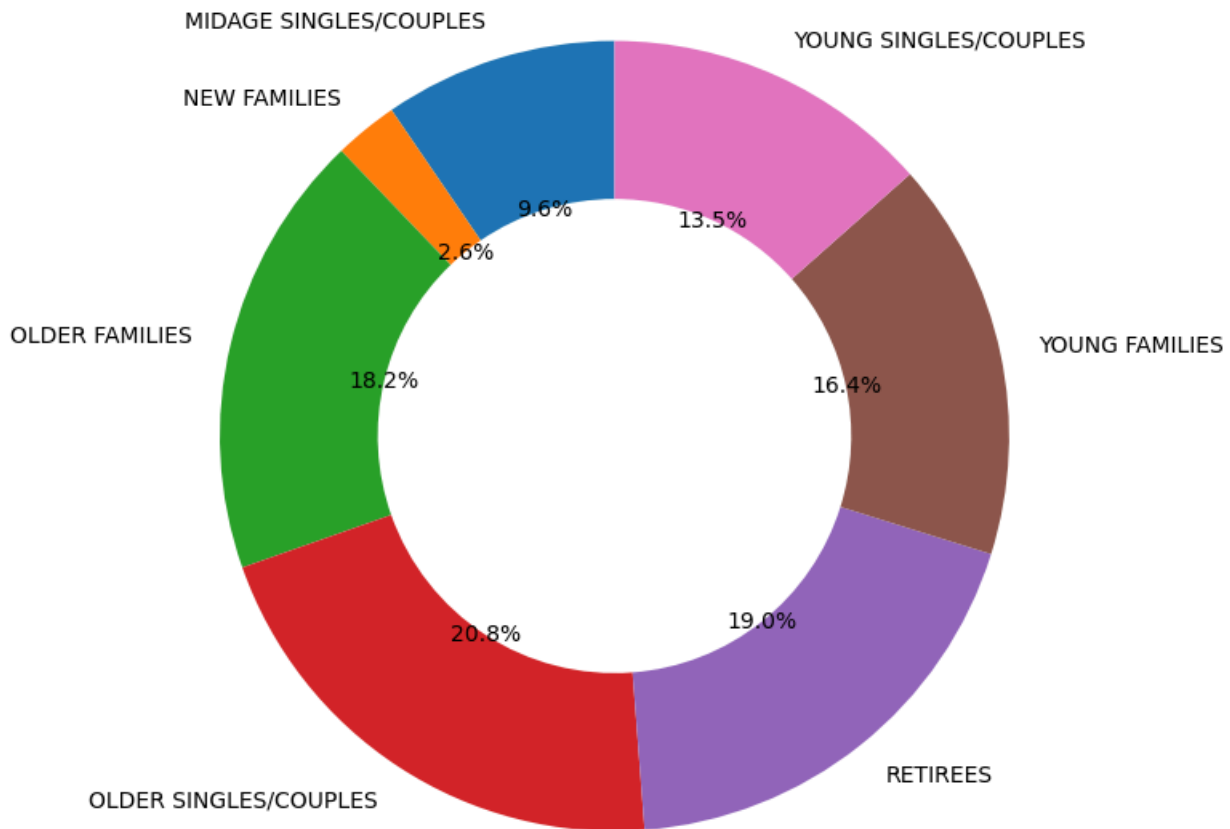
```
# Plot
plt.figure(figsize=(12,6))
plt.plot(monthly_sales.index, monthly_sales.values, marker='o')
plt.title('Total Sales per Month')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.grid(True)
plt.show()
```



```
In [63]: # Group sales by lifestage
sales_by_lifestage = merged.groupby('LIFESTAGE')['TOT_SALES'].sum()

# Plot as donut chart
plt.figure(figsize=(8,8))
plt.pie(sales_by_lifestage,
        labels=sales_by_lifestage.index,
        autopct='%1.1f%%',
        startangle=90,
        wedgeprops={'width':0.4}) # width <1 makes it a donut
plt.title('Share of Total Sales by Lifestage')
plt.show()
```


Share of Total Sales by Lifestage



```
In [ ]: #Insights:

#Older Families and Retirees are the top spenders on chips;
#Young and Midage Singles/Couples spend less but prefer mainstream brands.
#Premium vs. Mainstream: Mainstream customers dominate total sales across all life stages; premium customers are smaller
#Product Preferences: Most purchases are 150-175 g packs, showing a clear optimal size; strong brand loyalty to Smith's,
#Shopping Behaviour: Average ~2 packs and $7.3 per transaction; occasional bulk buying indicates opportunity for larger p
#Opportunities: Target families/retirees with promotions and family packs; upsell mainstream customers to premium offerin
```

```
In [1]: #Arkaprava Roy
#25.09.2025
```

```
In [2]: #Fetching the Data
```

```
In [8]: import pandas as pd
```

```
path = "QVI_transaction_data.xlsx"
df_raw = pd.read_excel(path, sheet_name="in")

print(df_raw.shape)
print(df_raw.columns.tolist())
df_raw.head()
```

```
(264836, 8)
```

```
['DATE', 'STORE_NBR', 'LYLTY_CARD_NBR', 'TXN_ID', 'PROD_NBR', 'PROD_NAME', 'PROD_QTY', 'TOT_SALES']
```

```
Out[8]:
```

	DATE	STORE_NBR	LYLTY_CARD_NBR	TXN_ID	PROD_NBR	PROD_NAME	PROD_QTY	TOT_SALES
0	43390	1	1000	1	5	Natural Chip Compny SeaSalt175g	2	6.0
1	43599	1	1307	348	66	CCs Nacho Cheese 175g	3	6.3
2	43605	1	1343	383	61	Smiths Crinkle Cut Chips Chicken 170g	2	2.9
3	43329	2	2373	974	69	Smiths Chip Thinly S/Cream&Onion 175g	5	15.0
4	43330	2	2426	1038	108	Kettle Tortilla ChpsHny&Jlpno Chili 150g	3	13.8

```
In [4]: #Basic Cleaning
```

```
In [9]: df = df_raw.rename(columns=str.strip).copy()
df["DATE"] = pd.to_datetime(df["DATE"], errors="coerce")
df = df.dropna(subset=["DATE", "STORE_NBR"])
df["STORE_NBR"] = pd.to_numeric(df["STORE_NBR"], errors="coerce").astype("Int64")
df["LYLTY_CARD_NBR"] = pd.to_numeric(df["LYLTY_CARD_NBR"], errors="coerce").astype("Int64")
df["TXN_ID"] = pd.to_numeric(df["TXN_ID"], errors="coerce").astype("Int64")
df["PROD_QTY"] = pd.to_numeric(df["PROD_QTY"], errors="coerce")
df["TOT_SALES"] = pd.to_numeric(df["TOT_SALES"], errors="coerce")
```

```
In [10]: #Validations and filters
```

```
df = df.query("PROD_QTY > 0 and TOT_SALES > 0")
df = df.drop_duplicates(subset=["STORE_NBR", "TXN_ID", "PROD_NBR", "DATE"])
```

```
In [11]: #Creating Monthly keys
```

```
df["year_month"] = df["DATE"].values.astype("datetime64[M]")
```

```
In [12]: #Isolationg Control Store
```

```
trial_stores = {77, 86, 88}

control_candidates = df.loc[~df["STORE_NBR"].isin(trial_stores), "STORE_NBR"].dropna().unique().tolist()
len(control_candidates), control_candidates[:10]
```

```
Out[12]: (269, [1, 2, 4, 5, 7, 8, 9, 13, 19, 20])
```

```
In [14]: df["year_month"] = df["DATE"].values.astype("datetime64[M]")
controls_df = df[df["STORE_NBR"].isin(control_candidates)].copy()
```

```
control_monthly = (
    controls_df.groupby(["STORE_NBR", "year_month"]).agg(
        total_revenue=("TOT_SALES", "sum"),
        total_transactions=("TXN_ID", "nunique"),
        total_customers=("LYLTY_CARD_NBR", "nunique")
    ).reset_index()
)

control_monthly["txns_per_customer"] = (
    control_monthly["total_transactions"] / control_monthly["total_customers"].replace(0, pd.NA)
)
control_monthly["revenue_per_txn"] = (
    control_monthly["total_revenue"] / control_monthly["total_transactions"].replace(0, pd.NA)
)

control_monthly = control_monthly.sort_values(["STORE_NBR", "year_month"])
control_monthly.head()
```

Out[14]:

	STORE_NBR	year_month	total_revenue	total_transactions	total_customers	txns_per_customer	revenue_per_txn
0	1	1970-01-01	2393.60	572	345	1.657971	4.184615
1	2	1970-01-01	2005.80	505	313	1.613419	3.971881
2	3	1970-01-01	12802.45	1489	364	4.090659	8.598019
3	4	1970-01-01	14647.65	1667	379	4.398417	8.786833
4	5	1970-01-01	9500.80	1358	238	5.705882	6.996171

In [15]:

```
# Ensure correct month key based on real dates
df = df.copy()
df["DATE"] = pd.to_datetime(df["DATE"], errors="coerce")
df = df.dropna(subset=["DATE"])
df["year_month"] = df["DATE"].dt.to_period("M").dt.to_timestamp()
```

In [16]:

```
import pandas as pd
import numpy as np

trial_stores = {77, 86, 88}
trial_df = df[df["STORE_NBR"].isin(trial_stores)].copy()

trial_monthly = (
    trial_df.groupby(["STORE_NBR", "year_month"]).agg(
        total_revenue=("TOT_SALES", "sum"),
        total_transactions=("TXN_ID", "nunique"),
        total_customers=("LYLTY_CARD_NBR", "nunique")
    ).reset_index()
)

trial_monthly["txns_per_customer"] = (
    trial_monthly["total_transactions"] / trial_monthly["total_customers"].replace(0, np.nan)
)
trial_monthly["revenue_per_txn"] = (
    trial_monthly["total_revenue"] / trial_monthly["total_transactions"].replace(0, np.nan)
)

trial_monthly = trial_monthly.sort_values(["STORE_NBR", "year_month"])
trial_monthly.to_csv("trial_monthly.csv", index=False)
trial_monthly.head()
```

Out[16]:

	STORE_NBR	year_month	total_revenue	total_transactions	total_customers	txns_per_customer	revenue_per_txn
0	77	1970-01-01	3040.00	562	356	1.578652	5.409253
1	86	1970-01-01	10635.35	1526	273	5.589744	6.969430
2	88	1970-01-01	16333.25	1857	388	4.786082	8.795504

In [17]:

```
# Fix the month key
df["DATE"] = pd.to_datetime(df["DATE"], errors="coerce")
df = df.dropna(subset=["DATE"])
df["year_month"] = df["DATE"].dt.to_period("M").dt.to_timestamp()

# Rebuild control and trial monthly tables
trial_stores = {77, 86, 88}
control_candidates = df.loc[~df["STORE_NBR"].isin(trial_stores), "STORE_NBR"].dropna().unique()

def make_monthly(sub):
    out = (sub.groupby(["STORE_NBR", "year_month"])
        .agg(total_revenue=("TOT_SALES", "sum"),
            total_transactions=("TXN_ID", "nunique"),
            total_customers=("LYLTY_CARD_NBR", "nunique"))
        .reset_index())
    out["txns_per_customer"] = out["total_transactions"] / out["total_customers"].replace(0, pd.NA)
    out["revenue_per_txn"] = out["total_revenue"] / out["total_transactions"].replace(0, pd.NA)
    return out.sort_values(["STORE_NBR", "year_month"])

control_monthly = make_monthly(df[df["STORE_NBR"].isin(control_candidates)].copy())
trial_monthly = make_monthly(df[df["STORE_NBR"].isin(trial_stores)].copy())

control_monthly.to_csv("control_monthly.csv", index=False)
trial_monthly.to_csv("trial_monthly.csv", index=False)
```

In [19]:

```
# Auto-detect pre-trial and trial windows from available months
all_months = df["year_month"].dropna().sort_values().unique()
n_months = len(all_months)

if n_months >= 9:
```

```

pre_n, trial_m = 6, 3
elif n_months >= 6:
    pre_n, trial_m = n_months-3, 3
elif n_months >= 4:
    pre_n, trial_m = n_months-2, 2
else:
    pre_n, trial_m = max(n_months-1, 1), 1 # minimal split

pre_months = all_months[:pre_n]
trial_months = all_months[-trial_m:]

print(f"Months available: {n_months}")
print("Pre-trial:", pre_months)
print("Trial:", trial_months)

```

```

Months available: 1
Pre-trial: <DatetimeArray>
['1970-01-01 00:00:00']
Length: 1, dtype: datetime64[ns]
Trial: <DatetimeArray>
['1970-01-01 00:00:00']
Length: 1, dtype: datetime64[ns]

```

```

In [20]: # Ensure DATE has real calendar values
df["DATE"] = pd.to_datetime(df["DATE"], dayfirst=True, errors="coerce") # set dayfirst if needed
# Verify min/max actual dates
print(df["DATE"].min(), df["DATE"].max())

df["year_month"] = df["DATE"].dt.to_period("M").dt.to_timestamp()
print(df["year_month"].dropna().sort_values().unique()[12])

```

```

1970-01-01 00:00:00.000043282 1970-01-01 00:00:00.000043646
<DatetimeArray>
['1970-01-01 00:00:00']
Length: 1, dtype: datetime64[ns]

```

```

In [21]: import pandas as pd
import numpy as np

# If reading from Excel, keep using the existing load:
df = pd.read_excel("QVI_transaction_data.xlsx", sheet_name="in")

# Convert Excel serial date to real calendar date
df["DATE"] = pd.to_datetime(df["DATE"], origin="1899-12-30", unit="D", errors="coerce")

# Quick sanity check
print("Date range:", df["DATE"].min(), "→", df["DATE"].max())
print(df["DATE"].dt.to_period("M").unique()[6])

# Month key for aggregation
df = df.dropna(subset=["DATE"])
df["year_month"] = df["DATE"].dt.to_period("M").dt.to_timestamp()

```

```

Date range: 2018-07-01 00:00:00 → 2019-06-30 00:00:00
<PeriodArray>
['2018-10', '2019-05', '2018-08', '2019-06', '2018-09', '2019-02']
Length: 6, dtype: period[M]

```

```

In [22]: import pandas as pd
import numpy as np

# ----- 1) Load and date conversion -----
path = "QVI_transaction_data.xlsx" # update if needed
sheet = "in" # the sheet detected earlier

df = pd.read_excel(path, sheet_name=sheet)
df = df.rename(columns=str.strip)

# Convert Excel serial date (e.g., 43390) to real calendar date
df["DATE"] = pd.to_datetime(df["DATE"], origin="1899-12-30", unit="D", errors="coerce")

# Basic filters and types
df = df.dropna(subset=["DATE", "STORE_NBR"])
for c in ["STORE_NBR", "LYLT_CARD_NBR", "TXN_ID", "PROD_NBR"]:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce").astype("Int64")
df["PROD_QTY"] = pd.to_numeric(df.get("PROD_QTY"), errors="coerce")
df["TOT_SALES"] = pd.to_numeric(df.get("TOT_SALES"), errors="coerce")

# Remove impossible values and duplicate lines per txn
df = df.query("PROD_QTY > 0 and TOT_SALES > 0")
df = df.drop_duplicates(subset=["STORE_NBR", "TXN_ID", "PROD_NBR", "DATE"])

```

```

# Month key
df["year_month"] = df["DATE"].dt.to_period("M").dt.to_timestamp()

print("Date span:", df["DATE"].min(), "→", df["DATE"].max())
print("Months:", pd.Index(df["year_month"].unique()).sort_values()[:6].tolist())

# ----- 2) Build monthly aggregates -----
trial_stores = {77, 86, 88}
control_candidates = (
    df.loc[~df["STORE_NBR"].isin(trial_stores), "STORE_NBR"]
    .dropna().unique().tolist()
)

def make_monthly(sub):
    out = (sub.groupby(["STORE_NBR", "year_month"]).agg(
        total_revenue=("TOT_SALES", "sum"),
        total_transactions=("TXN_ID", "nunique"),
        total_customers=("LYLTY_CARD_NBR", "nunique")
    ).reset_index())
    out["txns_per_customer"] = out["total_transactions"] / out["total_customers"].replace(0, pd.NA)
    out["revenue_per_txn"] = out["total_revenue"] / out["total_transactions"].replace(0, pd.NA)
    return out.sort_values(["STORE_NBR", "year_month"])

control_monthly = make_monthly(df[df["STORE_NBR"].isin(control_candidates)].copy())
trial_monthly = make_monthly(df[df["STORE_NBR"].isin(trial_stores)].copy())

control_monthly.to_csv("control_monthly.csv", index=False)
trial_monthly.to_csv("trial_monthly.csv", index=False)

print("control_monthly rows:", len(control_monthly), "stores:", control_monthly['STORE_NBR'].nunique())
print("trial_monthly rows:", len(trial_monthly), "stores:", trial_monthly['STORE_NBR'].nunique())

# ----- 3) Auto-detect pre-trial and trial windows -----
all_months = (
    pd.Index(control_monthly["year_month"])
    .append(pd.Index(trial_monthly["year_month"]))
    .unique().sort_values()
)

n = len(all_months)
if n >= 9:
    pre_n, trial_m = 6, 3
elif n >= 6:
    pre_n, trial_m = n-3, 3
elif n >= 4:
    pre_n, trial_m = n-2, 2
else:
    pre_n, trial_m = max(n-1, 1), 1

pre_months = all_months[:pre_n]
trial_months = all_months[-trial_m:]

print("Detected months:", n)
print("Pre-trial months:", list(pre_months))
print("Trial months:", list(trial_months))

```

Date span: 2018-07-01 00:00:00 → 2019-06-30 00:00:00

Months: [Timestamp('2018-07-01 00:00:00'), Timestamp('2018-08-01 00:00:00'), Timestamp('2018-09-01 00:00:00'), Timestamp('2018-10-01 00:00:00'), Timestamp('2018-11-01 00:00:00'), Timestamp('2018-12-01 00:00:00')]

control_monthly rows: 3133 stores: 269

trial_monthly rows: 36 stores: 3

Detected months: 12

Pre-trial months: [Timestamp('2018-07-01 00:00:00'), Timestamp('2018-08-01 00:00:00'), Timestamp('2018-09-01 00:00:00'), Timestamp('2018-10-01 00:00:00'), Timestamp('2018-11-01 00:00:00'), Timestamp('2018-12-01 00:00:00')]

Trial months: [Timestamp('2019-04-01 00:00:00'), Timestamp('2019-05-01 00:00:00'), Timestamp('2019-06-01 00:00:00')]

```

In [ ]: import numpy as np
import pandas as pd

pre_months = pd.to_datetime(["2018-07-01", "2018-08-01", "2018-09-01", "2018-10-01", "2018-11-01", "2018-12-01"])
trial_months = pd.to_datetime(["2019-04-01", "2019-05-01", "2019-06-01"])
metrics = ["total_revenue", "total_customers", "txns_per_customer"]

def scale01(x):
    x = np.asarray(x, dtype=float)
    a = np.nanmin(x)
    b = np.nanmax(x)
    if not np.isfinite(a) or not np.isfinite(b) or b <= a:
        return np.zeros_like(x)
    return (x - a) / (b - a)

def similarity_score(trial_store, control_store):

```

```

base = pd.DataFrame({"year_month": pre_months})

ts = (
    trial_monthly.loc[(trial_monthly["STORE_NBR"]==trial_store) &
        (trial_monthly["year_month"].isin(pre_months)),
        ["year_month"]+metrics]
    .copy()
)
cs = (
    control_monthly.loc[(control_monthly["STORE_NBR"]==control_store) &
        (control_monthly["year_month"].isin(pre_months)),
        ["year_month"]+metrics]
    .copy()
    .rename(columns={m: f"{m}_c" for m in metrics})
)

# Outer join ensures columns exist even if months are missing
aligned = base.merge(ts, on="year_month", how="left").merge(cs, on="year_month", how="left")

corrs, mags = [], []
for m in metrics:
    a = aligned[m]
    b = aligned[f"{m}_c"]
    if a.notna().sum() >= 2 and b.notna().sum() >= 2:
        r = a.corr(b)
        corrs.append((r + 1)/2 if pd.notna(r) else 0.0)
    else:
        corrs.append(0.0)

    d = np.abs(a.to_numpy(dtype=float) - b.to_numpy(dtype=float))
    mags.append(float(np.nanmean(1.0 - scale01(d))) if np.isfinite(d).any() else 0.0)

return float(np.nanmean([np.nanmean(corrs), np.nanmean(mags)]))

def select_best_control(trial_store, candidate_ids, top_k=1):
    scores = []
    for cid in candidate_ids:
        try:
            s = similarity_score(trial_store, cid)
        except Exception:
            s = -np.inf
        scores.append((cid, s))
    scores = [x for x in scores if np.isfinite(x[1])]
    scores.sort(key=lambda x: x[1], reverse=True)
    return scores[:top_k]

trial_ids = [77, 86, 88]
candidate_ids = sorted(control_monthly["STORE_NBR"].unique().tolist())

best_controls = {t: select_best_control(t, candidate_ids, top_k=1) for t in trial_ids}
print(best_controls)

```

```

In [25]: from scipy import stats

def compare_trial_vs_control(trial_store, control_store):
    t_df = trial_monthly.query("STORE_NBR == @trial_store and year_month.isin(@trial_months)").copy()
    c_df = control_monthly.query("STORE_NBR == @control_store and year_month.isin(@trial_months)").copy()

    # Align months
    base = pd.DataFrame({"year_month": trial_months})
    t_df = base.merge(t_df, on="year_month", how="left")
    c_df = base.merge(c_df, on="year_month", how="left", suffixes=("", "_c"))

    # Relative lift (%)
    lift_rev = (t_df["total_revenue"].sum() - c_df["total_revenue"].sum()) / c_df["total_revenue"].sum()
    lift_cus = (t_df["total_customers"].sum() - c_df["total_customers"].sum()) / c_df["total_customers"].sum()
    lift_tpc = (t_df["txns_per_customer"].mean() - c_df["txns_per_customer"].mean()) / c_df["txns_per_customer"].mean()

    # Welch t-tests across trial months (n=3)
    p_rev = stats.ttest_ind(t_df["total_revenue"], c_df["total_revenue"], equal_var=False, nan_policy="omit").pvalue
    p_cus = stats.ttest_ind(t_df["total_customers"], c_df["total_customers"], equal_var=False, nan_policy="omit").pvalue
    p_tpc = stats.ttest_ind(t_df["txns_per_customer"], c_df["txns_per_customer"], equal_var=False, nan_policy="omit").pvalue

    return {
        "lift_revenue_pct": float(lift_rev*100),
        "lift_customers_pct": float(lift_cus*100),
        "lift_txns_per_cust_pct": float(lift_tpc*100),
        "p_revenue": float(p_rev),
        "p_customers": float(p_cus),
        "p_txns_per_cust": float(p_tpc),
    }

```

```

In [26]: # 1) Control selection (uses the fixed functions defined earlier)
trial_ids = [77, 86, 88]
candidate_ids = sorted(control_monthly["STORE_NBR"].unique().tolist())

best_controls = {t: select_best_control(t, candidate_ids, top_k=1) for t in trial_ids}
print("Best control per trial:", best_controls)

# 2) Trial-period comparison for each trial-control pair
from scipy import stats
trial_months = pd.to_datetime(["2019-04-01", "2019-05-01", "2019-06-01"])

def compare_trial_vs_control(trial_store, control_store):
    t_df = trial_monthly.query("STORE_NBR == @trial_store and year_month.isin(@trial_months)").copy()
    c_df = control_monthly.query("STORE_NBR == @control_store and year_month.isin(@trial_months)").copy()
    base = pd.DataFrame({"year_month": trial_months})
    t_df = base.merge(t_df, on="year_month", how="left")
    c_df = base.merge(c_df, on="year_month", how="left", suffixes=("", "_c"))

    # Lifts
    lift_rev = (t_df["total_revenue"].sum() - c_df["total_revenue"].sum()) / c_df["total_revenue"].sum()
    lift_cus = (t_df["total_customers"].sum() - c_df["total_customers"].sum()) / c_df["total_customers"].sum()
    lift_tpc = (t_df["txns_per_customer"].mean() - c_df["txns_per_customer"].mean()) / c_df["txns_per_customer"].mean()

    # Welch t-tests across months (n=3)
    p_rev = stats.ttest_ind(t_df["total_revenue"], c_df["total_revenue"], equal_var=False, nan_policy="omit").pvalue
    p_cus = stats.ttest_ind(t_df["total_customers"], c_df["total_customers"], equal_var=False, nan_policy="omit").pvalue
    p_tpc = stats.ttest_ind(t_df["txns_per_customer"], c_df["txns_per_customer"], equal_var=False, nan_policy="omit").pvalue

    return {
        "trial_store": trial_store,
        "control_store": control_store,
        "lift_revenue_pct": round(float(lift_rev*100), 2),
        "lift_customers_pct": round(float(lift_cus*100), 2),
        "lift_txns_per_cust_pct": round(float(lift_tpc*100), 2),
        "p_revenue": float(p_rev),
        "p_customers": float(p_cus),
        "p_txns_per_cust": float(p_tpc)
    }

results = []
for t in trial_ids:
    c = best_controls[t][0][0]
    results.append(compare_trial_vs_control(t, c))

pd.DataFrame(results)

```

```

c /= stddev[:, None]
C:\Users\User\OneDrive\Documents\Anaconda\Lib\site-packages\numpy\lib\function_base.py:2898: RuntimeWarning: invalid value encountered in divide
  c /= stddev[:, None, :]
C:\Users\User\AppData\Local\Temp\ipykernel_19104\2379582694.py:13: RuntimeWarning: Engine has switched to 'python' because numexpr does not support extension array dtypes. Please set your engine to python manually.
  t_df = trial_monthly.query("STORE_NBR == @trial_store and year_month.isin(@trial_months)").copy()
C:\Users\User\AppData\Local\Temp\ipykernel_19104\2379582694.py:14: RuntimeWarning: Engine has switched to 'python' because numexpr does not support extension array dtypes. Please set your engine to python manually.
  c_df = control_monthly.query("STORE_NBR == @control_store and year_month.isin(@trial_months)").copy()
Best control per trial: {77: [(3, 0.7131906723998402)], 86: [(247, 0.7236924248656127)], 88: [(178, 0.6727789907454113)]}
C:\Users\User\AppData\Local\Temp\ipykernel_19104\2379582694.py:13: RuntimeWarning: Engine has switched to 'python' because numexpr does not support extension array dtypes. Please set your engine to python manually.
  t_df = trial_monthly.query("STORE_NBR == @trial_store and year_month.isin(@trial_months)").copy()
C:\Users\User\AppData\Local\Temp\ipykernel_19104\2379582694.py:14: RuntimeWarning: Engine has switched to 'python' because numexpr does not support extension array dtypes. Please set your engine to python manually.
  c_df = control_monthly.query("STORE_NBR == @control_store and year_month.isin(@trial_months)").copy()
C:\Users\User\AppData\Local\Temp\ipykernel_19104\2379582694.py:13: RuntimeWarning: Engine has switched to 'python' because numexpr does not support extension array dtypes. Please set your engine to python manually.
  t_df = trial_monthly.query("STORE_NBR == @trial_store and year_month.isin(@trial_months)").copy()
C:\Users\User\AppData\Local\Temp\ipykernel_19104\2379582694.py:14: RuntimeWarning: Engine has switched to 'python' because numexpr does not support extension array dtypes. Please set your engine to python manually.
  c_df = control_monthly.query("STORE_NBR == @control_store and year_month.isin(@trial_months)").copy()

```

```

Out[26]:
   trial_store  control_store  lift_revenue_pct  lift_customers_pct  lift_txns_per_cust_pct  p_revenue  p_customers  p_txns_per_cust
0           77              3         -71.99          -52.81             -12.00    0.000930     0.002146     0.018818
1           86             247          -9.79           -4.95             -7.28    0.145209     0.148551     0.148435
2           88             178          53.62           21.22             -0.84    0.026586     0.082306     0.804589

```

```

In [27]: #Observations
#Store 77 vs control 3: Total revenue is down about 72%, driven primarily by fewer unique customers (-53%) and fewer tran
#Store 86 vs control 247: Revenue is down about 10%, with small declines in customers (-5%) and transactions per customer
#Store 88 vs control 178: Revenue is up about 54%, with most of the lift explained by more customers (+21%) rather than t

```

```

In [28]: #Visualization

import pandas as pd
import matplotlib.pyplot as plt

# Assume `results_df` is the DataFrame you printed (trial_store, control_store, lift_* and p-values)
results_df = pd.DataFrame(results) # or reuse the existing variable

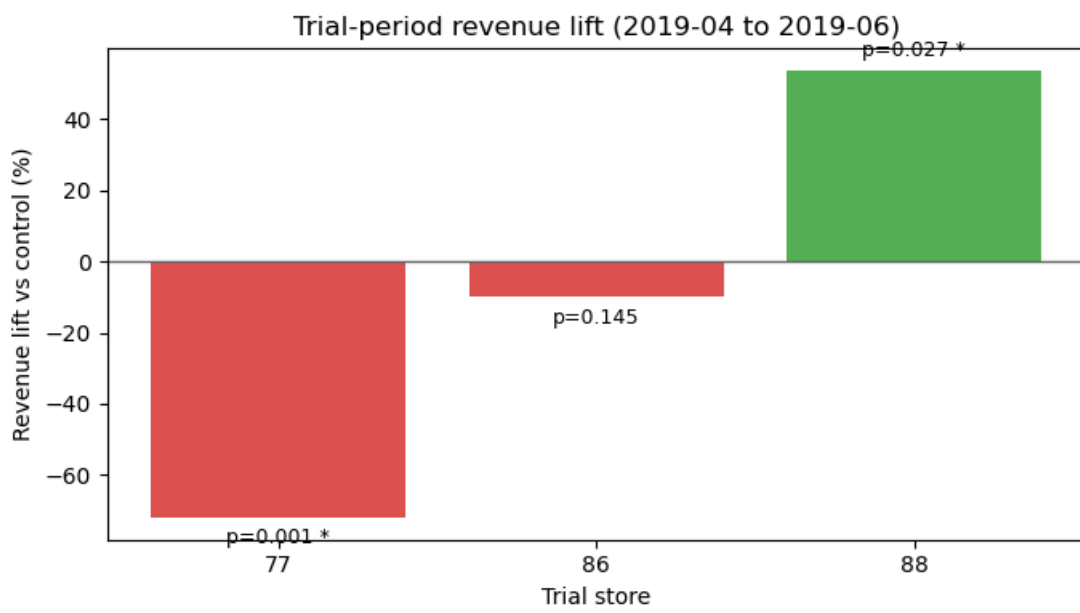
fig, ax = plt.subplots(figsize=(7,4))
stores = results_df["trial_store"].astype(str)
lift = results_df["lift_revenue_pct"]

colors = lift.apply(lambda x: "#2ca02c" if x > 0 else "#d62728")
bars = ax.bar(stores, lift, color=colors, alpha=0.8)

# Significance annotation
for i, row in results_df.iterrows():
    ax.text(i, row["lift_revenue_pct"] + (3 if row["lift_revenue_pct"] >= 0 else -3),
            (f"p={:.3f}".format(row["p_revenue"]) + (" *" if row["p_revenue"] < 0.05 else "")),
            ha="center", va="bottom" if row["lift_revenue_pct"] >= 0 else "top", fontsize=9)

ax.axhline(0, color="#666", lw=1)
ax.set_ylabel("Revenue lift vs control (%)")
ax.set_xlabel("Trial store")
ax.set_title("Trial-period revenue lift (2019-04 to 2019-06)")
plt.tight_layout()
plt.show()

```

In []: